

SOFTWARE ENGINEERING PROJECT REPORT

Stockly

Inventory & Supply Chain Management System

*A Full-Stack Role-Based Warehouse, Order, Invoice, and Analytics Platform
Built with Next.js 16, TypeScript 5, Prisma ORM 6.4, and MongoDB*

Version: 2.0.1

Date: April 8, 2026

Framework: Next.js 16 (App Router), React 19

Language: TypeScript 5 (strict mode)

Database: MongoDB via Prisma ORM 6.4

Live Demo: <https://stockly-inventory.vercel.app>

License: MIT Open Source

Report Type: SRS + Architecture + White-Box Testing

Test Cases: 107 across 10 modules

Coverage Target: 85% Statements / 90% Branch

This document covers the complete Software Requirements Specification, system design, implementation details, white-box testing strategy, deployment, and future roadmap.

Abstract

Stockly is a production-grade, role-based inventory and supply chain management platform that consolidates product management, order processing, invoice generation, warehouse tracking, supplier coordination, customer support, and business analytics into a single cohesive web application. The system serves three distinct user roles — administrators, suppliers, and clients — each with tailored dashboards and access controls enforced at both the API and UI layers.

This report presents the complete software engineering documentation for Stockly v2.0.1. It covers the Software Requirements Specification (SRS) with functional and non-functional requirements, system architecture and data model design, implementation details of critical algorithms and integrations, a comprehensive white-box testing strategy with 107 documented test cases across 10 modules, deployment procedures for both local and cloud environments, and a future development roadmap.

The platform is built on a modern full-stack architecture: **Next.js 16** with the App Router paradigm for server-side rendering and API routes, **TypeScript 5** in strict mode for end-to-end type safety, **Prisma ORM 6.4** with **MongoDB** as the persistence layer, and **JWT-based authentication** with **bcryptjs** for secure session management. Optional integrations include Stripe (payments), Shippo (shipping), Brevo (transactional email), ImageKit (media), Upstash Redis (caching and rate limiting), Sentry (error monitoring), and OpenRouter (AI insights).

The core business logic encompasses product lifecycle management with SKU tracking and QR code generation, a multi-state order management system with atomic stock reservation semantics, invoice generation with financial integrity guarantees (including negative-total prevention via `Math.max(0, ...)`), demand forecasting using statistical moving averages and z-score anomaly detection, and a comprehensive RBAC system. White-box testing targets 85% statement coverage and 90% branch coverage across all critical modules, with special emphasis on security-critical paths (JWT verification, password hashing), financial logic (invoice totals, stock reservation), and edge cases (null coalescing, boundary values).

Key outcomes: The system reduces manual inventory tracking overhead, provides real-time stock visibility, automates order-to-invoice workflows, and delivers actionable business intelligence through integrated analytics and AI-assisted forecasting.

Introduction

Project Overview

Stockly is a production-grade, full-stack inventory and supply chain management system developed to address the operational challenges faced by small-to-medium enterprises (SMEs) engaged in product distribution, warehousing, and retail. The platform consolidates product management, order processing, invoice generation, supplier coordination, warehouse tracking, customer support, and business analytics into a single, cohesive web application accessible from any modern browser.

The system is publicly accessible at <https://stockly-inventory.vercel.app> and is open-sourced under the MIT license. It is built with a technology stack chosen for developer productivity, runtime performance, and long-term maintainability.

Unlike traditional inventory tools that serve a single user type, Stockly provides **three fully distinct role-based experiences** within a single application:

- **Admin (Store Owner):** Complete system control — products, orders, invoices, warehouses, users, analytics, and portal oversight of both clients and suppliers.
- **Supplier:** Scoped view of their own products, associated orders, revenue metrics, low-stock alerts, and support tickets.
- **Client:** A consumer-facing portal for browsing the product catalog, placing orders, paying invoices via Stripe, and managing support requests.

The application follows the **Next.js 16 App Router** paradigm, where both the frontend (React Server Components and Client Components) and the backend (API Route Handlers) coexist within the same codebase. This architecture eliminates the need for a separate backend server, simplifies deployment, and enables efficient server-side rendering (SSR) and static generation (SSG) where appropriate.

Purpose

The primary purpose of Stockly is to eliminate the inefficiencies of manual or spreadsheet-based inventory management by providing a unified digital platform with the following capabilities:

1. **Real-time stock visibility:** Automatic computation of available stock using the formula $\text{availableStock} = \text{quantity} - \text{reservedQuantity}$, with status labels (available, stock low, out of stock) updated dynamically.
2. **Automated order-to-invoice workflows:** Orders automatically trigger invoice creation, status tracking, and payment collection, reducing administrative overhead and human error.
3. **Financial integrity:** Invoice calculations use `Math.max(0, subtotal + tax + shipping - discount)` to prevent negative totals, and nullish coalescing (`??`) ensures explicit zero values are respected over fallback values.
4. **Demand forecasting:** Statistical analysis of historical sales data (moving averages, standard deviation, linear regression) predicts future stock requirements and generates reorder recommendations.
5. **Multi-role access control:** JWT-based authentication with role-based query filtering ensures data isolation between clients, suppliers, and administrators.
6. **Operational transparency:** Comprehensive audit logs, activity history, and import history provide a full trail of all system changes.
7. **Extensible integrations:** The system is designed so the core application runs with only three environment variables, with all integrations (Stripe, Shippo, Brevo, etc.) being optional and gracefully degraded when not configured.

Scope

The scope of this project encompasses the following functional domains:

Domain	Scope Description
Product Management	Full CRUD with SKU, categories, suppliers, stock levels, status, QR codes, CSV/Excel import/export, image upload via ImageKit
Order Management	Multi-item orders with tax/shipping/discount, 6-state status machine, stock reservation/release logic
Invoice Management	Auto-generated invoices linked 1:1 to orders, PDF export, email delivery, Stripe payment links, overdue tracking
Warehouse Management	CRUD for warehouses, stock allocation per warehouse, stock transfer between warehouses
User Management	Admin-controlled user CRUD, role assignment (admin/supplier/client/user/retailer), activity tracking
Supplier Portal	Supplier-scoped dashboard with product/order/revenue views, low-stock alerts
Client Portal	Consumer catalog with browse/filter, order placement, invoice payment, support tickets
Analytics	Revenue charts, order trends, category breakdowns, demand forecasting, AI insights via OpenRouter
Support System	Ticket creation, reply threads, priority/status management, role-scoped visibility
Product Reviews	Customer reviews with admin moderation (approve/reject), eligibility verification by order
Notifications	In-app notification system, optional email via Brevo with QStash queue
API Infrastructure	RESTful API with JWT auth, Zod

Domain	Scope Description
	validation, rate limiting, Redis caching, OpenAPI spec

Issues and Challenges

1. **Race Condition in Order Number Generation:** `generateOrderNumber()` uses a non-atomic count-based sequence. Under high concurrency, duplicate sequence numbers are theoretically possible. Mitigation: use a distributed atomic counter (e.g., Redis INCR) in production.
2. **Invoice Number Collision Risk:** `generateInvoiceNumber()` uses a random 4-digit suffix with a while-loop uniqueness check. Collision probability is 1/9000 per attempt, but the loop is unbounded under extreme load.
3. **JWT Secret Default Fallback:** If `JWT_SECRET` is not set, the system defaults to `"your_jwt_secret"` — a critical security vulnerability in production.
4. **Non-Atomic Stock Reservation:** Stock reservation during order creation involves multiple sequential Prisma updates. A server crash between order creation and stock reservation could leave stock in an inconsistent state. MongoDB transactions should be used for full atomicity.
5. **Forecasting Data Dependency:** The demand forecasting engine requires at least 7 days of historical sales data for trend analysis. Products with fewer data points receive a default “stable” classification.
6. **Optional Integration Degradation:** When Stripe, Shippo, Brevo, or Redis are not configured, the system falls back to mock/disabled behavior, which may not be immediately obvious to end users without proper UI indicators.

Technology Stack Summary

Layer	Technology	Purpose
Framework	Next.js 16 (App Router)	SSR, SSG, API routes, file-based routing
Language	TypeScript 5 (strict)	End-to-end type safety
Database	MongoDB + Prisma ORM 6.4	Document storage, schema validation, query builder
Auth	JWT + bcryptjs	Stateless session management, password hashing
State (server)	TanStack Query	Server state, caching, background refetch
State (client)	Zustand	Lightweight client-side state

Layer	Technology	Purpose
UI	Tailwind CSS + shadcn/ui	Utility-first styling, accessible component primitives
Forms	React Hook Form + Zod	Form management and schema validation
Payments	Stripe	Checkout sessions, payment intents, webhooks
Shipping	Shippo	Label generation, rate calculation, tracking
Email	Brevo + QStash	Transactional email, background job queue
Images	ImageKit	CDN-backed image upload and transformation
Cache	Upstash Redis	API response caching, rate limiting
Monitoring	Sentry	Error tracking and performance monitoring
AI	OpenRouter	AI-powered business insights

Requirement Analysis

Software Requirements Specification (SRS)

This chapter presents the formal Software Requirements Specification for Stockly. Requirements are classified as Functional (FR) or Non-Functional (NFR) and prioritized using the MoSCoW method (Must Have, Should Have, Could Have, Won't Have).

Stakeholders and User Roles

Stakeholder	Role	Primary Concerns
Store Owner	Admin	Full system control, analytics, user management, financial oversight
Product Supplier	Supplier	Own product visibility, order tracking, revenue reporting
Customer	Client	Product browsing,

Stakeholder	Role	Primary Concerns
		order placement, invoice payment, support
System Admin	Admin	User management, system configuration, audit logs
Developer	Internal	API access, integration management, monitoring

Functional Requirements

Authentication and User Management

- **FR-AUTH-01** The system SHALL allow users to register with email, name, and password. Passwords must be hashed using bcrypt with a minimum of 10 salt rounds before storage.
- **FR-AUTH-02** The system SHALL authenticate users via email/password and issue a signed JWT token stored as an httpOnly cookie named `session_id` with a 1-hour expiry.
- **FR-AUTH-03** The system SHALL support Google OAuth 2.0 as an alternative authentication method.
- **FR-AUTH-04** The system SHALL validate JWT tokens on every protected API request. Expired, malformed, or tampered tokens SHALL result in a 401 Unauthorized response.
- **FR-AUTH-05** The system SHALL support session logout by clearing the `session_id` cookie.
- **FR-AUTH-06** The system SHALL enforce role-based access control. Users with insufficient roles SHALL receive a 403 Forbidden response.
- **FR-AUTH-07** Admin users SHALL be able to create, update, deactivate, and delete user accounts and assign roles.

Product Management

- **FR-PROD-01** The system SHALL support full CRUD operations for products with fields: name, SKU (unique), price, quantity, reservedQuantity, status, categoryId, supplierId, description, imageUrl, qrCodeUrl, and expirationDate.
- **FR-PROD-02** Product status SHALL be automatically computed: `available` (quantity $\geq$ threshold), `stock_low` (quantity $\leq$ threshold), `out_of_stock` (quantity ≤ 0).

- **FR-PROD-03** The system SHALL generate QR codes for products and store them via ImageKit.
- **FR-PROD-04** The system SHALL support bulk product import from CSV and Excel files, with per-row error reporting and import history tracking.
- **FR-PROD-05** The system SHALL support product image upload via ImageKit with automatic cleanup of old images on update.
- **FR-PROD-06** SKU values SHALL be globally unique across all products.
- **FR-PROD-07** The system SHALL support product filtering by category, supplier, status, and price range, with pagination.

Order Management

- **FR-ORD-01** The system SHALL support creation of orders with multiple line items, each capturing productId, quantity, unit price (snapshot), and subtotal.
- **FR-ORD-02** On order creation, the system SHALL verify available stock ($\text{quantity} - \text{reservedQuantity} \geq \text{requestedQuantity}$) for each item and reject the order if any item has insufficient stock.
- **FR-ORD-03** On successful order creation, the system SHALL increment reservedQuantity for each ordered product.
- **FR-ORD-04** The order status state machine SHALL support the following transitions: pending → confirmed → processing → shipped → delivered, with cancellation possible from any non-delivered state.
- **FR-ORD-05** On order confirmation/payment, the system SHALL decrement both quantity and reservedQuantity for each ordered product.
- **FR-ORD-06** On order cancellation from a pending state, the system SHALL decrement reservedQuantity (releasing the reservation). On cancellation from confirmed/processing/shipped, the system SHALL increment quantity (restoring stock).
- **FR-ORD-07** Order totals SHALL be computed as:
total = subtotal + tax + shipping – discount.
- **FR-ORD-08** The system SHALL generate unique order numbers in the format ORD-YYYY-MMDD-HHMMSS-NNNN.
- **FR-ORD-09** Clients SHALL only see their own orders. Suppliers SHALL only see orders containing their products. Admins SHALL see all orders.

Invoice Management

- **FR-INV-01** Each order SHALL have at most one invoice (enforced by a unique constraint on orderId).

- **FR-INV-02** Invoice numbers SHALL be unique and generated in the format INV-YYYYMMDD-HHMMSS-XXXX with collision detection.
- **FR-INV-03** Invoice totals SHALL use `Math.max(0, subtotal + tax + shipping - discount)` to prevent negative totals.
- **FR-INV-04** Tax, shipping, and discount values SHALL use nullish coalescing (`??`) to respect explicit zero values from the request over order defaults.
- **FR-INV-05** New invoices SHALL always start with `status = "draft"` and `amountPaid = 0`.
- **FR-INV-06** The system SHALL support invoice PDF generation and email delivery to clients.
- **FR-INV-07** The system SHALL generate Stripe payment links for invoices and process payments via Stripe Checkout.
- **FR-INV-08** The system SHALL send automated overdue invoice reminders via a scheduled endpoint protected by `INTERNAL_API_KEY`.

Warehouse and Stock Allocation

- **FR-WH-01** The system SHALL support CRUD operations for warehouses with fields: name, address, type, and status.
- **FR-WH-02** The system SHALL support stock allocation records linking products to warehouses with per-warehouse quantity and reservedQuantity.
- **FR-WH-03** The system SHALL support stock transfer operations between warehouses with status tracking (pending, completed, cancelled).
- **FR-WH-04** A product-warehouse combination SHALL be unique (enforced by a composite unique index on `[productId, warehouseId]`).

Analytics and Forecasting

- **FR-ANA-01** The admin dashboard SHALL display: total products, orders, invoices, revenue, low-stock count, and recent activity.
- **FR-ANA-02** The business insights page SHALL display revenue trends, order volume charts, category breakdowns, and supplier performance metrics.
- **FR-ANA-03** The demand forecasting engine SHALL analyze the last 30 days of sales data and forecast the next 14 days using weighted moving averages.
- **FR-ANA-04** The forecasting engine SHALL detect sales anomalies using z-score analysis with a threshold of 2.0 standard deviations.
- **FR-ANA-05** The system SHALL generate reorder recommendations: urgent (≤ 7 days until stockout), soon (≤ 14 days), overstocked (> 90 days), normal (otherwise).

- **FR-ANA-06** The system SHALL compute a confidence score (0–100) for each forecast based on data volume and coefficient of variation.
- **FR-ANA-07** The system SHALL optionally provide AI-generated business insights via OpenRouter when OPENROUTER_API_KEY is configured.

Support Tickets and Product Reviews

- **FR-SUP-01** Any authenticated user SHALL be able to create support tickets with subject, description, priority (low/medium/high/urgent), and optional links to products, orders, or suppliers.
- **FR-SUP-02** Support tickets SHALL support threaded replies from any authenticated user.
- **FR-SUP-03** Ticket status SHALL follow: open → in_progress → resolved → closed.
- **FR-REV-01** Clients SHALL be able to submit product reviews (rating 1–5, comment) only for products they have ordered and received.
- **FR-REV-02** Product reviews SHALL require admin approval before being publicly visible.
- **FR-REV-03** The system SHALL enforce one review per order-product combination.

Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses SHALL complete within 500ms for 95% of requests under normal load. Dashboard aggregation queries SHALL complete within 2 seconds.
NFR-02	Security	All passwords SHALL be hashed with bcrypt (min 10 rounds). JWT tokens SHALL expire after 1 hour. All API routes SHALL validate authentication before processing.
NFR-03	Security	The session_id cookie SHALL be httpOnly, Secure (in production), and

ID	Category	Requirement
NFR-04	Rate Limiting	SameSite=Strict to prevent XSS and CSRF attacks. API endpoints SHALL enforce rate limiting via Upstash Redis. Exceeded limits SHALL return HTTP 429 with a Retry-After header.
NFR-05	Availability	The system SHALL target 99.9% uptime when deployed on Vercel with MongoDB Atlas.
NFR-06	Scalability	The caching layer (Upstash Redis) SHALL reduce database load by caching frequently accessed resources (products, orders, dashboard stats) with appropriate TTLs.
NFR-07	Maintainability	All API routes SHALL use Zod schemas for input validation. Validation errors SHALL return structured error messages with field-level details.
NFR-08	Auditability	All create, update, and delete operations on core entities SHALL be recorded in the AuditLog collection with userId, action, entityType, entityId, and timestamp.
NFR-09	Accessibility	The UI SHALL follow WCAG 2.1 AA guidelines using accessible shadcn/ui

ID	Category	Requirement
NFR-10	Portability	components built on Radix UI primitives. The system SHALL run on any Node.js 18+ environment with only three required environment variables (DATABASE_URL, JWT_SECRET, NEXT_PUBLIC_API_URL).
NFR-11	Observability	When Sentry is configured, all unhandled exceptions SHALL be captured with full stack traces and user context.
NFR-12	Data Integrity	MongoDB unique indexes SHALL enforce uniqueness on Product.sku, Invoice.invoiceNumber, Order.orderNumber, User.email, and StockAllocation[productId, warehouseId].

Use Cases

Use Case: Place an Order (Client)

Actor: Client

Precondition: Client is authenticated. Products are in stock.

Main Flow:

1. Client browses the product catalog and selects items.
2. Client adds items to the order with desired quantities.
3. System validates stock availability for each item.
4. System creates the order, reserves stock, and generates an order number.
5. System creates a linked invoice in draft status.

6. Client receives order confirmation with invoice details.

Alternate Flow (Insufficient Stock): System returns an error message specifying the product and available quantity. Order is not created. No stock is reserved.

Postcondition: Order exists in pending status. reservedQuantity incremented for each ordered product.

Use Case: Process Payment (Client)

Actor: Client

Precondition: Invoice exists in sent or draft status. Stripe is configured.

Main Flow:

1. Client navigates to invoice detail page.
2. Client clicks “Pay Now” button.
3. System creates a Stripe Checkout session with the invoice amount.
4. Client is redirected to Stripe’s hosted payment page.
5. Client completes payment.
6. Stripe sends a checkout.session.completed webhook to /api/payments/webhook.
7. System updates invoice status to paid, sets paidAt, and updates order paymentStatus to paid.

Postcondition: Invoice status is paid. Order payment status is paid.

Use Case: Manage Inventory (Admin)

Actor: Admin

Main Flow:

1. Admin navigates to the Products section.
2. Admin can create, edit, or delete products.
3. Admin can import products in bulk via CSV/Excel upload.
4. System validates each row, imports valid rows, and reports errors for invalid rows.
5. Import history is recorded with success/failure counts.
6. Admin can view low-stock products and trigger reorder workflows.

Design and Architecture

System Architecture Overview

Stockly follows a **monolithic full-stack architecture** using the Next.js 16 App Router, where the frontend and backend coexist in a single deployable unit. This approach simplifies deployment, eliminates cross-origin complexity, and enables efficient server-side rendering.

Project Directory Structure

The project follows a well-organized directory structure that separates concerns clearly:

```
stock-inventory/
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # Root layout, metadata, providers
│   ├── page.tsx                       # Home (store overview for admin)
│   ├── login/ register/              # Auth pages
│   ├── products/ orders/ invoices/   # Core resource pages
│   ├── categories/ suppliers/ warehouses/
│   ├── client/                       # Client portal dashboard
│   ├── supplier/                     # Supplier portal dashboard
│   ├── support-tickets/              # Support tickets (all roles)
│   ├── business-insights/           # Business charts (admin)
│   ├── admin/                        # Admin panel (nested routes)
│   └── api/                          # API route handlers
├── components/                       # React components
│   ├── ui/                           # shadcn/ui primitives
│   ├── layouts/                      # Navbar, AdminSidebar,
│   └── PageWithSidebar
│       ├── admin/                    # Admin-only content components
│       └── [domain]/                 # Domain-specific components
├── lib/                              # Shared backend/utility code
│   ├── api/                          # API client, CORS, rate limit
│   ├── email/                        # Brevo templates and queue
│   ├── forecasting/                  # Demand forecasting engine
│   ├── server/                       # Server-side data aggregation
│   ├── validations/                 # Zod schemas
│   ├── stripe/ shippo/              # Payment and shipping
│   └── env.ts                        # Environment validation
├── prisma/                           # MongoDB models
│   ├── schema.prisma                 # Prisma singleton
│   ├── client.ts                     # Prisma singleton
│   └── *.ts                          # Repository helpers
├── utils/                            # auth.ts (JWT), auth.client.ts
├── hooks/queries/                    # TanStack Query hooks
├── contexts/                         # Auth context
├── types/                           # TypeScript type definitions
└── middleware.ts                     # Next.js middleware (auth
    redirects)
```

Database Design

Data Models

The database uses MongoDB with Prisma ORM. All models use MongoDB ObjectId as the primary key. The following describes the core data models:

Model	Key Fields	Description
User	id, email, password, role, googleId	Authentication, roles (admin/supplier/client/user/retailer), email preferences
Product	id, sku, quantity, reservedQuantity, status	Inventory items; availableStock = quantity - reservedQuantity
Order	id, orderNumber, status, paymentStatus, total	Purchase orders with 6-state status machine
OrderItem	id, orderId, productId, price, quantity	Line items with price snapshot at order time
Invoice	id, invoiceNumber, orderId, status, total, amountDue	Financial documents, 1:1 with Order
Category	id, name, status, userId	Product classification
Supplier	id, name, status, userId	Product suppliers
Warehouse	id, name, address, type, status	Storage locations
StockAllocation	id, productId, warehouseId, quantity	Per-warehouse stock levels; unique on [productId, warehouseId]
StockTransfer	id, productId, fromWarehouseId, toWarehouseId, status	Stock movement between warehouses
SupportTicket	id, subject, status, priority, userId	Customer support requests
ProductReview	id, productId, userId, rating, status	Customer reviews with moderation
Notification	id, userId, type, title, read	In-app notifications
AuditLog	id, userId, action, entityType, entityId	Full audit trail

Model	Key Fields	Description
ImportHistory	id, userId, importType, successRows, failedRows	Bulk import tracking
SystemConfig	id, key, value, type, category	Admin-managed app settings

Key Relationships

- **Order** ↔ **Invoice**: One-to-one relationship enforced by @unique on Invoice.orderId. Cascade delete: deleting an order deletes its invoice.
- **Order** ↔ **OrderItem**: One-to-many. Each order has multiple line items. Cascade delete on order.
- **OrderItem** ↔ **Product**: Many-to-one. Each order item references a product but stores a price snapshot to preserve historical accuracy.
- **User** ↔ **Notification**: One-to-many. Cascade delete on user.
- **SupportTicket** ↔ **SupportTicketReply**: One-to-many. Cascade delete on ticket.
- **StockAllocation**: Composite unique index on [productId, warehouseId] ensures one allocation record per product-warehouse pair.

API Design

Authentication Flow

API Endpoint Reference

Method + Path	Auth	Description
POST /api/auth/register	Public	Register new user
POST /api/auth/login	Public	Login, set session cookie
POST /api/auth/logout	Auth	Clear session cookie
GET /api/auth/session	Auth	Get current session user
GET POST /api/products	Auth	List / create products
GET PATCH DELETE /api/products/[id]	Auth	Get / update / delete product
GET /api/products/import	Admin	Bulk import from CSV/Excel
GET POST /api/orders	Auth	List / create orders

Method + Path	Auth	Description
GET PATCH DELETE /api/orders/[id]	Auth	Get / update / delete order
GET POST /api/invoices	Auth	List / create invoices
GET /api/invoices/[id]/pdf	Auth	Generate invoice PDF
POST /api/invoices/[id]/send	Auth	Send invoice email
POST /api/payments/checkout	Auth	Create Stripe Checkout session
POST /api/payments/webhook	Public	Stripe webhook handler
GET /api/dashboard	Admin	Admin dashboard stats
GET /api/portal/client	Client	Client dashboard
GET /api/portal/supplier	Supplier	Supplier dashboard
GET /api/forecasting	Admin	Demand forecasting data
GET /api/ai/insights	Admin	AI business insights
GET POST /api/support-tickets	Auth	List / create tickets
GET POST /api/users	Admin	List / create users
GET /api/audit-logs	Admin	Audit log entries
GET PATCH /api/system-config	Admin	System configuration
GET /api/health	Public	Health check
GET /api/openapi	Public	OpenAPI specification

Frontend Architecture

Component Hierarchy

The frontend follows a layered component architecture:

- **Layout Components:** Navbar, AdminSidebar, PageWithSidebar — provide the structural shell for each role's experience.

- **Page Components:** Located in components/Pages/ — orchestrate data fetching and compose domain components.
- **Domain Components:** Located in components/[domain]/ — e.g., ProductList, OrderDialog, InvoiceTable — handle domain-specific UI and interactions.
- **UI Primitives:** Located in components/ui/ — shadcn/ui components (Button, Dialog, Table, Badge, etc.) built on Radix UI for accessibility.
- **Shared Components:** ErrorBoundary, PaginationSelector, LoadingSpinner — reused across domains.

State Management

State Type	Solution
Server state (API data)	TanStack Query (React Query) — handles caching, background refetch, optimistic updates
Client UI state	Zustand stores — lightweight, no boilerplate, used for modals, filters, selections
Form state	React Hook Form — uncontrolled inputs, Zod resolver for validation
Auth state	React Context (AuthContext) — provides current user to all components
Theme state	ThemeProvider with localStorage persistence

Data Fetching Pattern

All data fetching uses TanStack Query hooks defined in hooks/queries/. Each hook wraps an Axios call to the corresponding API endpoint:

```
// hooks/queries/useProducts.ts
export const useProducts = (filters: ProductFilters) => {
  return useQuery({
    queryKey: ['products', filters],
    queryFn: () => apiClient.get('/api/products', { params:
filters }),
    staleTime: 30_000, // 30 seconds
  });
};
```

Cache invalidation is handled centrally via lib/react-query/invalidate-all.ts, which provides typed invalidation helpers for each resource type.

Implementation Details

Authentication System

JWT Token Management

The authentication system is implemented in `utils/auth.ts` (server-side) and `utils/auth.client.ts` (client-side). The server-side module uses the `jsonwebtoken` library with dynamic import to prevent bundling on the client.

```
// Token generation
export const generateToken = (userId: string): string => {
  return jwt.sign({ userId }, JWT_SECRET, { expiresIn: "1h" });
};

// Token verification with 5 distinct branches
export const verifyToken = (token: string): { userId: string } | null
=> {
  // Branch A: falsy, "null", or "undefined" string
  if (!token || token === "null" || token === "undefined") return
  null;
  // Branch B: client-side execution guard
  if (typeof window !== "undefined") return null;
  // Branch C: jwt module unavailable
  const jwt = require("jsonwebtoken");
  if (!jwt?.verify) return null;
  try {
    // Branch E: successful verification
    const decoded = jwt.verify(token, JWT_SECRET) as { userId:
string };
    return { userId: decoded.userId };
  } catch {
    // Branch D: expired, invalid, or malformed token
    return null;
  }
};
```

The client-side module (`utils/auth.client.ts`) avoids direct JWT operations entirely, instead making an API call to `/api/auth/session` with `credentials: "include"` to let the server verify the `httpOnly` cookie:

```
export const getSessionClient = async (): Promise<User | null> => {
  try {
    const response = await fetch("/api/auth/session", {
      method: "GET",
      credentials: "include", // sends httpOnly session_id cookie
    });
    return response.ok ? await response.json() : null;
  } catch {
    return null;
  }
};
```

```
}  
};
```

Password Security

Passwords are hashed using bcrypt with 10 salt rounds before storage. The hashPassword and comparePassword functions provide a clean abstraction:

```
export const hashPassword = async (password: string): Promise<string>  
=> {  
  const salt = await bcrypt.genSalt(10);  
  return bcrypt.hash(password, salt);  
};
```

```
export const comparePassword = async (  
  password: string,  
  hashedPassword: string  
) : Promise<boolean> => {  
  return bcrypt.compare(password, hashedPassword);  
};
```

Order Management and Stock Reservation

Stock Reservation State Machine

The order management system implements a critical stock reservation pattern to prevent overselling. The available stock formula is:

$$\text{availableStock} = \text{quantity} - \text{reservedQuantity}$$

- **Order CREATE (pending):** reservedQuantity **INCREASES** — stock is reserved but not yet deducted.
- **Order CONFIRM / PAY:** quantity **DECREASES**, reservedQuantity **DECREASES** — stock is physically consumed.
- **Order CANCEL (was pending):** reservedQuantity **DECREASES** — reservation is released.
- **Order CANCEL (was confirmed/paid):** quantity **INCREASES** — stock is restored.

Order Creation Algorithm

```
async function createOrder(data, userId) {  
  const orderNumber = await generateOrderNumber();  
  const orderItemsData = [];  
  const productsToReserve = [];  
  
  // Validate all items BEFORE any DB writes (fail-fast)  
  for (const item of data.items) {  
    const product = await prisma.product.findUnique({  
      where: { id: item.productId }  
    });  
  }  
}
```

```

    });
    if (!product) throw new Error(`Product not found: ${
item.productId}`);

    const availableStock = product.quantity -
(product.reservedQuantity ?? 0);
    if (availableStock < item.quantity) {
        throw new Error(
            `Insufficient stock for ${product.name}. ` +
            `Available: ${availableStock}, Requested: ${item.quantity}`
        );
    }
    orderItemsData.push({ ...item, price: product.price });
    productsToReserve.push({ id: product.id, qty: item.quantity });
}

// Create order (single DB write)
const order = await prisma.order.create({ data: { orderNumber,
...data } });

// Reserve stock for each product
for (const { id, qty } of productsToReserve) {
    await prisma.product.update({
        where: { id },
        data: { reservedQuantity: { increment: qty } }
    });
}
return order;
}

```

Invoice Generation and Financial Logic

Invoice Number Generation

Invoice numbers use a random 4-digit suffix with a uniqueness check loop:

```

async function generateInvoiceNumber(): Promise<string> {
    let invoiceNumber: string;
    let exists: boolean;
    do {
        const datePart = format(new Date(), "yyyyMMdd");
        const timePart = format(new Date(), "HHmmss");
        const randomPart = Math.floor(1000 + Math.random() * 9000);
        invoiceNumber = `INV-${datePart}-${timePart}-${randomPart}`;
        exists = !(await prisma.invoice.findUnique({ where:
{ invoiceNumber } }));
    } while (exists);
    return invoiceNumber;
}

```

Financial Calculation Logic

```
// Nullish coalescing respects explicit 0 values from request
const tax      = data.tax      ?? order.tax      ?? 0;
const shipping = data.shipping ?? order.shipping ?? 0;
const discount = data.discount ?? order.discount ?? 0;

// Math.max(0, ...) prevents negative totals
const total = Math.max(0, subtotal + tax + shipping - discount);

// Invoice always starts unpaid
const amountPaid = 0;
const amountDue  = total;
```

The use of ?? (nullish coalescing) instead of || (logical OR) is critical: it ensures that an explicit `data.tax = 0` is respected and not overridden by `order.tax = 15`. With ||, zero would be treated as falsy and the fallback would incorrectly apply.

Demand Forecasting Engine

The forecasting engine (`lib/forecasting/demand-forecast.ts`) implements several statistical algorithms:

Moving Average Calculation

```
function calculateMovingAverage(values: number[], window: number):
number[] {
  return values.map((_, i) => {
    const start = Math.max(0, i - window + 1); // partial window for
early indices
    const windowValues = values.slice(start, i + 1);
    return windowValues.reduce((a, b) => a + b, 0) /
windowValues.length;
  });
}
```

Standard Deviation (Population)

```
function calculateStdDev(values: number[]): number {
  if (values.length === 0) return 0; // early return for empty array
  const mean = values.reduce((a, b) => a + b, 0) / values.length;
  const squaredDiffs = values.map(v => Math.pow(v - mean, 2));
  const avgSquaredDiff = squaredDiffs.reduce((a, b) => a + b, 0) /
values.length;
  return Math.sqrt(avgSquaredDiff); // population std dev (divides by
N, not N-1)
}
```

Predicted Daily Sales Formula

When at least 7 days of recent data are available, the system uses a weighted blend:

$$\text{predictedDailySales} = 0.7 \times \text{recentAvg} + 0.3 \times \text{historicalAvg}$$

This gives more weight to recent trends while anchoring to historical patterns.

Confidence Score Algorithm

```
let confidence = 50; // base score
if (dailySales.length >= 30) confidence += 30;
else if (dailySales.length >= 14) confidence += 20;
else if (dailySales.length >= 7) confidence += 10;

const cv = stdDev / averageDailySales; // coefficient of variation
if (cv < 0.3) confidence += 20; // low variability = high confidence
else if (cv < 0.5) confidence += 10;

confidence = Math.min(100, confidence); // cap at 100
```

Rate Limiting and Caching

Rate Limiting

Rate limiting is implemented via Upstash Redis using a sliding window algorithm. The `withRateLimit` middleware wraps API handlers:

```
// lib/api/rate-limit.ts
export async function withRateLimit(
  request: NextRequest,
  handler: () => Promise<NextResponse>
): Promise<NextResponse> {
  const ip = request.ip ?? "anonymous";
  const { success, limit, remaining, reset } = await
    ratelimit.limit(ip);
  if (!success) {
    return NextResponse.json(
      { error: "Too many requests" },
      { status: 429, headers: { "Retry-After": String(reset) } }
    );
  }
  return handler();
}
```

Redis Caching

The caching layer (`lib/cache/`) provides `getCache` and `setCache` utilities with configurable TTLs. Cache keys follow a namespaced pattern (e.g., `products:list:page1`, `dashboard:admin:userId`). Cache invalidation is triggered after any write operation via fire-and-forget calls.

Input Validation with Zod

All API inputs are validated using Zod schemas defined in `lib/validations/`. Each schema provides:

- Type coercion (e.g., string to number for query params)
- Field-level error messages
- Optional field handling with defaults
- Enum validation for status fields

```
export const createOrderSchema = z.object({
  clientId: z.string().optional(),
  items: z.array(z.object({
    productId: z.string().min(1, "Product ID required"),
    quantity: z.number().int().positive("Quantity must be positive"),
  })).min(1, "At least one item required"),
  tax: z.number().min(0).optional(),
  shipping: z.number().min(0).optional(),
  discount: z.number().min(0).optional(),
  notes: z.string().optional(),
});
```

Stripe Payment Integration

The Stripe integration handles two flows:

1. **Checkout Session Creation:** POST /api/payments/checkout creates a Stripe Checkout session with line items derived from the invoice. The client is redirected to Stripe's hosted payment page.
2. **Webhook Processing:** POST /api/payments/webhook receives Stripe events. The checkout.session.completed event triggers invoice status update to paid and order payment status update to paid. Webhook signature verification using STRIPE_WEBHOOK_SECRET prevents spoofed events.

Email Notification System

Email notifications are sent via Brevo (Sendinblue) with optional QStash queuing for background processing. The notification system supports:

- Invoice email delivery with PDF attachment
- Overdue invoice reminders (scheduled via INTERNAL_API_KEY-protected endpoint)
- Low-stock alerts
- Order status update notifications

Users can configure their email preferences per notification type via GET | PUT /api/user/email-preferences, stored as a JSON field on the User model.

Testing and Deployment

White-Box Testing Strategy

White-box testing (also called **structural testing**, **glass-box testing**, or **clear-box testing**) examines the *internal* logic, code paths, branches, loops, and data flows of an application — not just its external behaviour. It requires full knowledge of the source code and targets 100% statement, branch, and path coverage of critical modules.

Testing Goals

- Achieve **85% statement coverage** across all modules.
- Achieve **90% branch coverage** — every if/else branch tested.
- Validate all catch blocks are reachable and handled correctly.
- Verify all loops execute correctly for 0, 1, and N iterations.
- Confirm financial calculations produce mathematically correct results.
- Ensure security boundaries (JWT, RBAC) cannot be bypassed.

Types of White-Box Testing Applied

1. **Unit Testing** — Individual functions tested in isolation with mocked dependencies.
2. **Integration Testing** — API route handlers tested end-to-end with mocked Prisma.
3. **Control Flow Testing** — All branches and decision points covered.
4. **Data Flow Testing** — Tracking how data is created, transformed, and consumed.
5. **Boundary Value Analysis** — Testing edge values at limits (e.g., exact stock match, zero discount).
6. **Mutation Testing** — Verifying tests catch intentional code bugs.

Priority Classification

Priority	Label	Meaning
P1	Critical	Security, financial logic, auth guards. Must pass before any release.
P2	High	Core business logic, stock management, role-based access.
P3	Medium	Edge cases, format validation, confidence scores.
P4	Low	Informational,

Priority	Label	Meaning
		documentation-level tests.

Module 1: Authentication & Session Management

Source Files: `utils/auth.ts`, `utils/auth.client.ts`, `app/api/auth/login/route.ts`

verifyToken Branch Map

`verifyToken` has 5 distinct branches. All must be covered. The `JWT_SECRET` defaults to `"your_jwt_secret"` if the environment variable is not set — a **critical security risk** in production.

Branch	Condition	Result
A	token is falsy, "null", or "undefined"	return null
B	<code>typeof window !== "undefined"</code> (client-side)	return null
C	jwt module undefined or <code>jwt.verify</code> missing	return null
D	<code>jwt.verify</code> throws (expired / invalid / malformed)	catch → return null
E	<code>jwt.verify</code> succeeds	return { userId }

Function: `verifyToken` **Scenario:** Token is an empty string

Input: `token = ""`

Expected Path: Branch A (falsy check) → return null

Expected Output: null **Coverage:** Statement + Branch A

Function: `verifyToken` **Scenario:** Token is the string "null"

Input: `token = "null"`

Expected Path: Branch A (string "null" check) → return null

Expected Output: null **Coverage:** Branch A edge case

Function: `verifyToken` **Scenario:** Token is the string "undefined"

Input: `token = "undefined"`

Expected Path: Branch A → return null

Expected Output: null **Coverage:** Branch A edge case

Function: `verifyToken` **Scenario:** Valid JWT token on server side

Input: Token generated by `generateToken("user123")`

Expected Path: Branch E → return { userId: "user123" }

Expected Output: { userId: "user123" } **Coverage:** Happy path

Function: verifyToken **Scenario:** Expired JWT token

Input: Token with expiresIn: "1ms", verified after 10ms delay

Expected Path: Branch D → jwt.verify throws TokenExpiredError → return null

Expected Output: null **Coverage:** Exception handling branch

Function: verifyToken **Scenario:** Token signed with wrong secret

Input: jwt.sign({ userId: "x" }, "WRONG_SECRET")

Expected Path: Branch D → jwt.verify throws JsonWebTokenError → return null

Expected Output: null **Coverage:** Security branch

Function: verifyToken **Scenario:** Malformed token string

Input: token = "not.a.jwt.token"

Expected Path: Branch D → catch → return null

Expected Output: null **Coverage:** Malformed input

Function: generateToken **Scenario:** Valid userId generates decodable token

Input: userId = "abc123"

Expected: jwt.decode(result).userId === "abc123"

Coverage: Token payload integrity

Function: hashPassword + comparePassword **Scenario:** Round-trip verification

Input: password = "SecurePass123!"

Steps: (1) hash = await hashPassword(password) (2) result = await comparePassword(password, hash)

Expected: result === true **Coverage:** Round-trip password verification

Function: comparePassword **Scenario:** Wrong password against valid hash

Input: comparePassword("WrongPass", hashOf("CorrectPass"))

Expected: false **Coverage:** Negative comparison branch

Route: POST /api/auth/login **Scenario:** Missing email field

Input: { password: "test123" }

Expected Path: loginSchema.parse throws ZodError → 400

Expected Response: { error: "...", status: 400 }

Route: POST /api/auth/login **Scenario:** User not found

Input: { email: "nonexistent@test.com", password: "pass" }

Mock: prisma.user.findUnique returns null

Expected Response: { error: "Invalid email or password", status: 401 }

Route: POST /api/auth/login **Scenario:** User found but password is null

Mock: prisma.user.findUnique returns { id: "x", password: null }

Expected Path: !user.password → 500

Expected Response: { error: "User data corrupted: password missing", status: 500 }

Route: POST /api/auth/login **Scenario:** Correct credentials — full happy path

Input: { email: "admin@test.com", password: "correct" }

Mock: User found, bcrypt.compare returns true

Expected Response: Status **200**, Set-Cookie header with session_id

Function: getSessionFromRequest **Scenario:** No session_id cookie

Input: Request with no cookies

Expected Path: cookie === undefined → return null

Expected Output: null **Coverage:** Missing cookie branch

Function: getSessionFromRequest **Scenario:** Valid cookie but user deleted from DB

Mock: prisma.user.findUnique returns null

Expected Path: Decoded OK → DB query → null

Expected Output: null **Coverage:** Deleted user edge case

Module 2: Order Management & Stock State Machine

Source Files: prisma/order.ts, app/api/orders/route.ts,
app/api/orders/[id]/route.ts

Function: generateOrderNumber **Scenario:** First order of the day

Mock: prisma.order.count returns 0

Expected: Sequence = "0001", format: ORD-2026-0401-HHMMSS-0001

Coverage: Sequence generation, zero-count branch

Function: generateOrderNumber **Scenario:** 9999 orders already exist today

Mock: prisma.order.count returns 9999

Expected: Sequence = "10000" (5 digits, overflow of 4-digit pad)

Note: Race condition possible under high concurrency — not atomic

Coverage: Boundary value, sequence overflow

Function: createOrder **Scenario:** Product not found

Input: data.items = [{ productId: "nonexistent", quantity: 1 }]

Mock: prisma.product.findUnique returns null

Expected: throw Error("Product not found: nonexistent")

Coverage: Product not found branch

Function: createOrder **Scenario:** Insufficient available stock

Input: item.quantity = 10

Mock: product.quantity = 15, product.reservedQuantity = 8

Internal: availableStock = 15 - 8 = 7 < 10

Expected: throw Error("Insufficient stock. Available: 7, Requested: 10")

Coverage: Stock check branch (insufficient)

Function: createOrder **Scenario:** Exactly enough stock

Input: item.quantity = 7

Mock: product.quantity = 15, product.reservedQuantity = 8

Internal: availableStock = 7 == 7 (exactly enough)

Expected: Order created successfully, no error

Coverage: Boundary value (exact stock match)

Function: createOrder **Scenario:** One more than available stock

Input: item.quantity = 8

Mock: product.quantity = 15, product.reservedQuantity = 8

Internal: availableStock = 7 < 8

Expected: throw Error("Insufficient stock")

Coverage: Boundary value + 1

Function: createOrder **Scenario:** reservedQuantity is null

Mock: product.quantity = 10, product.reservedQuantity = null

Internal: availableStock = 10 - Number(null) = 10 - 0 = 10

Expected: availableStock correctly computed as 10

Coverage: Null coalescing in stock calculation

Function: createOrder **Scenario:** Multiple items, one fails stock check

Input: items = [{ productA, qty: 1 }, { productB, qty: 999 }]

Mock: productA has stock, productB has only 5

Expected: Error thrown on productB; NO reservation made for productA

Coverage: Partial failure rollback (fail-fast before any DB writes)

Function: createOrder **Scenario:** Tax, shipping, discount all provided

Input: subtotal=100, tax=10, shipping=5, discount=15

Expected: total = 100 + 10 + 5 - 15 = 100

Coverage: Total calculation formula

Function: createOrder **Scenario:** Tax and shipping are 0 (falsy)

Input: tax=0, shipping=0, discount=0

Internal: tax > 0 is false → stored as null

Expected: order.tax = null, order.shipping = null, order.discount = null

Coverage: Falsy-to-null conversion branches

Function: createOrder **Scenario:** Verify reservedQuantity incremented

Input: Single item, quantity = 3

Mock: product.reservedQuantity = 5 before order

Expected: After createOrder, product.reservedQuantity = 8

Coverage: Stock reservation side effect

Route: GET /api/orders **Scenario:** Unauthenticated request

Mock: getSessionFromRequest returns null

Expected: { error: "Unauthorized" }, status 401

Coverage: Auth guard branch

Route: GET /api/orders **Scenario:** Supplier with no supplier record

Mock: session.role = "supplier", getSupplierByUserId returns null

Expected: NextResponse.json([]) — empty array, not error

Coverage: Supplier without record branch

Route: GET /api/orders **Scenario:** Cache hit
Mock: getCache returns cached array
Expected: Returns cached data without DB query
Coverage: Cache hit branch

Route: GET /api/orders **Scenario:** Client role fetches only their orders
Mock: session.role = "client", session.id = "clientId"
Expected: getOrdersByClientId called with "clientId"
Coverage: Role-based query branching (client)

Route: GET /api/orders **Scenario:** Rate limit exceeded
Mock: withRateLimit returns a 429 response
Expected: 429 response returned immediately, no DB query
Coverage: Rate limit early return branch

Module 3: Invoice Generation & Financial Logic

Source Files: prisma/invoice.ts, app/api/invoices/route.ts

Function: generateInvoiceNumber **Scenario:** No collision on first attempt
Mock: prisma.invoice.findUnique returns null on first call
Expected: Returns "INV-YYYYMMDD-HHMMSS-XXXX" format; while loop executes once
Coverage: Happy path, no collision

Function: generateInvoiceNumber **Scenario:** First attempt collides, second unique
Mock: prisma.invoice.findUnique returns existing invoice on call 1, null on call 2
Expected: While loop executes twice, returns second generated number
Coverage: Collision handling branch (while loop iteration)

Function: generateInvoiceNumber **Scenario:** Verify format structure
Expected: Result matches regex /^INV-\d{8}-\d{6}-\d{4}\$/

Coverage: Output format validation

Function: createInvoice **Scenario:** Order not found
Mock: prisma.order.findUnique returns null
Expected: throw Error("Order with ID nonexistent not found")
Coverage: Order not found branch

Function: createInvoice **Scenario:** Invoice already exists for this order
Mock: prisma.invoice.findUnique returns existing invoice
Expected: throw Error("Invoice already exists for order {orderId}")
Coverage: Duplicate invoice prevention branch

Function: createInvoice **Scenario:** Total calculation with all fields
Input: order.subtotal=200, data.tax=20, data.shipping=10, data.discount=30
Expected: total = Math.max(0, 200 + 20 + 10 - 30) = 200
Coverage: Total formula

Function: createInvoice **Scenario:** Discount exceeds subtotal
Input: subtotal=50, tax=5, shipping=5, discount=100
Expected: total = Math.max(0, 50+5+5-100) = Math.max(0, -40) = 0
Coverage: Math.max(0, ...) negative prevention branch

Function: createInvoice **Scenario:** data.tax is 0 (explicit zero)
Input: data.tax = 0, order.tax = 15
Internal: tax = data.tax ?? order.tax ?? 0 = 0 ?? 15 = 0
Expected: tax = 0 (data value respected, NOT overridden by order.tax)
Coverage: Nullish coalescing vs OR operator distinction

Function: createInvoice **Scenario:** data.tax is undefined, order.tax is 15
Input: data.tax = undefined, order.tax = 15
Internal: tax = undefined ?? 15 ?? 0 = 15
Expected: tax = 15 (falls back to order.tax)
Coverage: Fallback chain in nullish coalescing

Function: createInvoice **Scenario:** data.tax and order.tax both null
Input: data.tax = null, order.tax = null
Internal: tax = null ?? null ?? 0 = 0
Expected: tax = 0 (final fallback)
Coverage: Double null fallback

Function: createInvoice **Scenario:** amountPaid always starts at 0
Expected: invoice.amountPaid = 0 regardless of input; invoice.amountDue = invoice.total
Coverage: Initial payment state

Function: createInvoice **Scenario:** Invoice status always starts as "draft"
Expected: invoice.status = "draft"
Coverage: Default status branch

Route: GET /api/invoices **Scenario:** Client role sees only their invoices
Mock: session.role = "client"
Expected: getInvoicesByClientId called (not getInvoicesByUser)
Coverage: Role-based query branch

Route: GET /api/invoices **Scenario:** Filter by multiple statuses
Input: ?status=draft&status=sent
Expected: filters.status = ["draft", "sent"]
Coverage: Multi-value query param parsing

Route: GET /api/invoices **Scenario:** No status filter provided
Expected: filters.status = undefined (not empty array)
Coverage: Empty array vs undefined branch

Module 4: Demand Forecasting Engine

Source File: lib/forecasting/demand-forecast.ts

Configuration Constants: ANALYSIS_DAYS = 30, FORECAST_DAYS = 14,
ANOMALY_THRESHOLD = 2.0, REORDER_LEAD_DAYS = 7

Function: calculateMovingAverage **Scenario:** Window larger than data length

Input: values = [10, 20, 30], window = 5

Expected: [10, 15, 20] (partial windows used for early elements)

Coverage: Math.max(0, i - window + 1) branch for early indices

Function: calculateMovingAverage **Scenario:** Window of 1

Input: values = [5, 10, 15], window = 1

Expected: [5, 10, 15] (each value is its own average)

Coverage: Window = 1 edge case

Function: calculateMovingAverage **Scenario:** Empty values array

Input: values = [], window = 3

Expected: [] (empty result)

Coverage: Empty input

Function: calculateStdDev **Scenario:** Empty array

Input: values = []

Expected: 0 (early return branch)

Coverage: Empty array branch

Function: calculateStdDev **Scenario:** All identical values (zero variance)

Input: values = [5, 5, 5, 5, 5]

Expected: 0 (population std dev of identical values)

Coverage: Zero variance branch

Function: calculateStdDev **Scenario:** Known values

Input: values = [2, 4, 4, 4, 5, 5, 7, 9]

Expected: 2.0 (population std dev)

Coverage: Standard calculation path

Function: analyzeTrend **Scenario:** Fewer than 7 data points

Input: dailySales.length = 5

Expected: { trend: "stable", percentageChange: 0 }

Coverage: Branch A (insufficient data)

Function: analyzeTrend **Scenario:** Strong upward trend

Input: 14 days of steadily increasing sales

Expected: percentageChange > 10, trend = "increasing"

Coverage: Branch B1 (increasing)

Function: analyzeTrend **Scenario:** Strong downward trend

Input: 14 days of steadily decreasing sales

Expected: percentageChange < -10, trend = "decreasing"

Coverage: Branch B2 (decreasing)

Function: detectAnomalies **Scenario:** stdDev is 0 (all identical values)

Input: All daily sales = 10

Expected: [] (no anomalies when stdDev = 0)

Coverage: Branch B (zero stdDev guard)

Function: detectAnomalies **Scenario:** Spike anomaly detected

Input: 30 days of sales with one day at 10x the mean

Expected: Anomaly with type = "spike", severity = "high" (zScore > 3)

Coverage: Branch C1 + C4

Function: generateProductForecast **Scenario:** Zero predicted daily sales

Input: All historical sales = 0

Expected: daysUntilStockout = null (no division by zero)

Coverage: predictedDailySales > 0 guard branch

Function: generateProductForecast **Scenario:** Urgent reorder recommendation

Input: availableStock = 10, predictedDailySales = 2

Internal: daysUntilStockout = floor(10/2) = 5 <= 7

Expected: reorderRecommendation = "urgent"

Coverage: Urgent reorder branch

Function: generateProductForecast **Scenario:** Overstocked recommendation

Input: availableStock = 1000, predictedDailySales = 2

Internal: daysUntilStockout = 500 > 90

Expected: reorderRecommendation = "overstocked"

Coverage: Overstocked branch

Function: generateProductForecast **Scenario:** Maximum confidence score

Input: 30+ days of data with CV < 0.3

Expected: confidenceScore = 100 (50 + 30 + 20 = 100, capped)

Coverage: Confidence score cap branch

Module 5: Email Notification System

Source Files: lib/email/notifications.ts, lib/queue/

Function: sendInvoiceEmail **Scenario:** Brevo not configured

Mock: BREVO_API_KEY is undefined

Expected: Function returns early without throwing; logs warning

Coverage: Missing config early return branch

Function: sendInvoiceEmail **Scenario:** User email preferences disable invoice emails

Mock: user.emailPreferences.invoiceEmails = false

Expected: Email not sent; function returns without calling Brevo API

Coverage: Email preference check branch

Function: sendInvoiceEmail **Scenario:** Brevo API call fails

Mock: Brevo API throws network error

Expected: Error caught and logged; function does not propagate exception

Coverage: Catch block in email send

Function: sendOverdueReminders **Scenario:** No overdue invoices

Mock: prisma.invoice.findMany returns []

Expected: No emails sent; function returns { sent: 0 }

Coverage: Empty result branch

Module 6: Rate Limiting & Caching

Source Files: lib/api/rate-limit.ts, lib/cache/

Function: withRateLimit **Scenario:** Redis not configured

Mock: UPSTASH_REDIS_URL is undefined

Expected: Rate limiting skipped; handler called directly

Coverage: Missing Redis config branch

Function: withRateLimit **Scenario:** Rate limit exceeded

Mock: ratelimit.limit returns { success: false, reset: 1234567890 }

Expected: Returns 429 with Retry-After header; handler NOT called

Coverage: Rate limit exceeded branch

Function: withRateLimit **Scenario:** Within rate limit

Mock: ratelimit.limit returns { success: true }

Expected: Handler called; response returned normally

Coverage: Rate limit pass branch

Function: getCache **Scenario:** Cache miss

Mock: Redis returns null for key

Expected: Returns null; no error thrown

Coverage: Cache miss branch

Function: getCache **Scenario:** Redis connection error

Mock: Redis throws connection error

Expected: Error caught; returns null (graceful degradation)

Coverage: Redis error catch branch

Module 7: Input Validation (Zod Schemas)

Source Files: lib/validations/product.ts, lib/validations/order.ts, lib/validations/invoice.ts

Schema: createProductSchema **Scenario:** Missing required field (name)

Input: { price: 10, quantity: 5, sku: "SKU-001" }

Expected: ZodError with path: ["name"], message: "Required"

Coverage: Required field validation

Schema: createProductSchema **Scenario:** Negative price

Input: { name: "Test", price: -5, quantity: 5, sku: "SKU-001" }

Expected: ZodError with path: ["price"], message about non-negative value
Coverage: Minimum value constraint

Schema: createOrderSchema **Scenario:** Empty items array

Input: { items: [] }

Expected: ZodError with path: ["items"], message about minimum 1 item

Coverage: Array minimum length constraint

Schema: createOrderSchema **Scenario:** Item quantity is 0

Input: { items: [{ productId: "x", quantity: 0 }] }

Expected: ZodError with path: ["items", 0, "quantity"]

Coverage: Positive integer constraint

Schema: loginSchema **Scenario:** Invalid email format

Input: { email: "not-an-email", password: "pass123" }

Expected: ZodError with path: ["email"], message about invalid email

Coverage: Email format validation

Schema: createInvoiceSchema **Scenario:** Invalid status value

Input: { status: "invalid_status" }

Expected: ZodError with enum validation error

Coverage: Enum constraint validation

Module 8: Role-Based Access Control (RBAC)

Source Files: app/api/users/route.ts, app/api/admin/*, middleware.ts

Route: GET /api/users **Scenario:** Non-admin user attempts access

Mock: session.role = "client"

Expected: { error: "Forbidden" }, status **403**

Coverage: Admin role guard branch

Route: GET /api/products **Scenario:** Supplier sees only their products

Mock: session.role = "supplier", session.supplierId = "sup123"

Expected: Query includes where: { supplierId: "sup123" }

Coverage: Supplier data isolation branch

Route: GET /api/invoices **Scenario:** Client sees only their invoices

Mock: session.role = "client", session.id = "client123"

Expected: Query includes where: { clientId: "client123" }

Coverage: Client data isolation branch

Middleware: middleware.ts **Scenario:** Unauthenticated access to protected route

Input: Request to /admin/products with no session cookie

Expected: Redirect to /login

Coverage: Auth redirect branch

Middleware: middleware.ts **Scenario:** Client tries to access admin route

Input: Request to /admin/dashboard with client session

Expected: Redirect to /client (client dashboard)

Coverage: Role-based redirect branch

Module 9: Data Integrity & Database Layer

Source Files: prisma/schema.prisma, prisma/product.ts, prisma/order.ts

Scenario: Attempt to create two products with the same SKU

Expected: Second creation throws Prisma unique constraint error

Coverage: Product.sku @unique constraint

Scenario: Attempt to create two invoices for the same order

Expected: Second creation throws Prisma unique constraint error on orderId

Coverage: Invoice.orderId @unique constraint

Scenario: Delete order cascades to order items and invoice

Expected: All related OrderItem and Invoice records deleted

Coverage: Cascade delete behavior

Scenario: StockAllocation composite unique constraint

Input: Create two allocations with same productId and warehouseId

Expected: Second creation throws unique constraint error

Coverage: @@unique([productId, warehouseId]) constraint

Scenario: User email uniqueness

Input: Register two users with the same email

Expected: Second registration throws unique constraint error

Coverage: User.email @unique constraint

Module 10: Stripe Payment & Webhook Handling

Source Files: app/api/payments/checkout/route.ts,
app/api/payments/webhook/route.ts

Route: POST /api/payments/webhook **Scenario:** Invalid webhook signature

Input: Request with tampered body or wrong stripe-signature header

Expected: stripe.webhooks.constructEvent throws; returns 400

Coverage: Signature verification failure branch

Route: POST /api/payments/webhook **Scenario:** checkout.session.completed event

Input: Valid Stripe event with type = "checkout.session.completed"

Expected: Invoice status updated to "paid"; order paymentStatus updated to "paid"

Coverage: Payment completion handler branch

Route: POST /api/payments/webhook **Scenario:** Unhandled event type

Input: Valid Stripe event with type = "customer.created"

Expected: Returns 200 without processing; no DB changes

Coverage: Unhandled event type branch

Route: POST /api/payments/checkout **Scenario:** Stripe not configured
Mock: STRIPE_API_KEY is undefined
Expected: Returns mock checkout response or 503 with clear error message
Coverage: Missing Stripe config branch

Route: POST /api/payments/checkout **Scenario:** Invoice not found
Input: { invoiceId: "nonexistent" }
Mock: prisma.invoice.findUnique returns null
Expected: Returns 404 with error message
Coverage: Invoice not found branch

Complete Test Case Matrix

Test ID	Module	Function/ Route	Priority	Branch
AUTH-001	Authenticati on	verifyToken	P1	A
AUTH-002	Authenticati on	verifyToken	P1	A
AUTH-003	Authenticati on	verifyToken	P1	A
AUTH-004	Authenticati on	verifyToken	P1	E
AUTH-005	Authenticati on	verifyToken	P1	D
AUTH-006	Authenticati on	verifyToken	P1	D
AUTH-007	Authenticati on	verifyToken	P1	D
AUTH-008	Authenticati on	generateTok en	P1	-
AUTH-009	Authenticati on	hashPasswor d + comparePass word	P1	-
AUTH-010	Authenticati on	comparePass word	P1	false
AUTH-011	Authenticati on	POST /api/auth/lo gin	P1	Zod
AUTH-012	Authenticati on	POST /api/auth/lo gin	P1	401
AUTH-013	Authenticati	POST	P2	500

Test ID	Module	Function/ Route	Priority	Branch
	on	/api/auth/login		
AUTH-014	Authentication	POST /api/auth/login	P1	200
AUTH-015	Authentication	getSessionFromRequest	P1	null
AUTH-016	Authentication	getSessionFromRequest	P2	null
ORD-001	Orders	generateOrderNumber	P2	seq
ORD-002	Orders	generateOrderNumber	P3	overflow
ORD-003	Orders	createOrder	P2	not found
ORD-004	Orders	createOrder	P2	insufficient
ORD-005	Orders	createOrder	P3	boundary
ORD-006	Orders	createOrder	P3	boundary+ 1
ORD-007	Orders	createOrder	P3	null coalesce
ORD-008	Orders	createOrder	P2	atomicity
ORD-009	Orders	createOrder	P2	total calc
ORD-010	Orders	createOrder	P3	falsy-null
ORD-011	Orders	createOrder	P2	reservation
ORD-012	Orders	GET /api/orders	P1	401
ORD-013	Orders	GET /api/orders	P2	supplier
ORD-014	Orders	GET /api/orders	P2	cache hit
ORD-015	Orders	GET /api/orders	P2	client role
ORD-016	Orders	GET /api/orders	P2	rate limit
INV-001	Invoices	generateInvoiceNumber	P2	no collision
INV-002	Invoices	generateInvoiceNumber	P3	collision
INV-003	Invoices	generateInvoiceNumber	P3	format

Test ID	Module	Function/ Route	Priority	Branch
		iceNumber		
INV-004	Invoices	createInvoice	P2	not found
INV-005	Invoices	createInvoice	P2	duplicate
INV-006	Invoices	createInvoice	P2	total calc
INV-007	Invoices	createInvoice	P2	negative total
INV-008	Invoices	createInvoice	P2	nullish 0
INV-009	Invoices	createInvoice	P3	nullish fallback
INV-010	Invoices	createInvoice	P3	double null
INV-011	Invoices	createInvoice	P1	amountPaid
INV-012	Invoices	createInvoice	P1	draft status
INV-013	Invoices	GET /api/invoices	P2	client role
INV-014	Invoices	GET /api/invoices	P3	multi-status
INV-015	Invoices	GET /api/invoices	P3	no filter
FORE-001	Forecasting	calculateMovingAverage	P3	partial window
FORE-002	Forecasting	calculateMovingAverage	P3	window=1
FORE-003	Forecasting	calculateMovingAverage	P3	empty
FORE-004	Forecasting	calculateStdDev	P3	empty
FORE-005	Forecasting	calculateStdDev	P3	zero variance
FORE-006	Forecasting	calculateStd	P3	known

Test ID	Module	Function/ Route	Priority	Branch
		Dev		values
FORE-007	Forecasting	analyzeTrend	P2	insufficient data
FORE-008	Forecasting	analyzeTrend	P2	increasing
FORE-009	Forecasting	analyzeTrend	P2	decreasing
FORE-010	Forecasting	detectAnomalies	P2	zero stdDev
FORE-011	Forecasting	detectAnomalies	P2	spike
FORE-012	Forecasting	generateProductForecast	P2	zero sales
FORE-013	Forecasting	generateProductForecast	P2	urgent
FORE-014	Forecasting	generateProductForecast	P2	overstocked
FORE-015	Forecasting	generateProductForecast	P3	max confidence
EMAIL-001	Email	sendInvoiceEmail	P2	no config
EMAIL-002	Email	sendInvoiceEmail	P2	pref disabled
EMAIL-003	Email	sendInvoiceEmail	P2	API error
EMAIL-004	Email	sendOverdueReminders	P2	empty result
RATE-001	Rate Limit	withRateLimit	P2	no Redis
RATE-002	Rate Limit	withRateLimit	P2	exceeded
RATE-003	Rate Limit	withRateLimit	P2	pass
CACHE-001	Cache	getCache	P3	miss
CACHE-002	Cache	getCache	P3	error
ZOD-001	Validation	createProductSchema	P2	required
ZOD-002	Validation	createProductSchema	P2	negative

Test ID	Module	Function/ Route	Priority	Branch
ZOD-003	Validation	ctSchema createOrderS chema	P2	price empty array
ZOD-004	Validation	createOrderS chema	P2	zero qty
ZOD-005	Validation	loginSchema	P3	email format
ZOD-006	Validation	createInvoic eSchema	P3	enum
RBAC-001	RBAC	GET /api/users	P1	403
RBAC-002	RBAC	GET /api/product s	P1	supplier isolation
RBAC-003	RBAC	GET /api/invoice s	P1	client isolation
RBAC-004	RBAC	middleware.t s	P1	auth redirect
RBAC-005	RBAC	middleware.t s	P1	role redirect
DB-001	Database	Product.sku	P1	unique
DB-002	Database	Invoice.orde rId	P1	unique
DB-003	Database	Order cascade	P2	cascade delete
DB-004	Database	StockAllocati on	P2	composite unique
DB-005	Database	User.email	P2	unique
STRIPE-001	Stripe	webhook	P1	sig verify
STRIPE-002	Stripe	webhook	P1	completed
STRIPE-003	Stripe	webhook	P2	unhandled
STRIPE-004	Stripe	checkout	P2	no config
STRIPE-005	Stripe	checkout	P2	not found

Code Coverage Requirements

Module	Critical Functions	Statement Target	Branch Target
Authentication	verifyToken,	100%	100%

Module	Critical Functions	Statement Target	Branch Target
Order Management	generateToken, hashPassword, comparePassword		
	createOrder, generateOrderNumber, updateOrderStatus	95%	95%
Invoice Generation	createInvoice, generateInvoiceNumber	95%	95%
Demand Forecasting	calculateMovingAverage, calculateStdDev, analyzeTrend, detectAnomalies	90%	90%
Email System	sendInvoiceEmail, sendOverdueReminders	85%	85%
Rate Limiting Caching	withRateLimit	90%	90%
	getCache, setCache, invalidateCache	85%	85%
Zod Validation	All schemas	90%	90%
RBAC	All role guards, middleware	100%	100%
Stripe Webhooks	constructEvent, handleCheckoutCompleted	95%	95%
Overall		85%	90%

Testing Tools and Setup

Recommended Testing Stack

Tool & Purpose & Usage
Vitest & Unit/Integration test

Tool & Purpose & Usage

runner & Fast, ESM-native, compatible with Next.js

@testing-library/react & Component testing & Test React components in isolation

msw (Mock Service Worker) & API mocking & Mock HTTP requests in tests

jest-mock-extended & Prisma mocking & Type-safe Prisma client mocks

@faker-js/faker & Test data generation & Generate realistic test fixtures

c8 / v8 & Code coverage & Statement, branch, function, line coverage

Running Tests

Run all tests once

```
npx vitest run
```

Run with coverage

```
npx vitest run --coverage
```

Run specific module

```
npx vitest run tests/auth.test.ts
```

Deployment

Local Development Setup

```
git clone <your-repo-url> && cd stock-inventory
```

```
npm install
```

```
cp .env.example .env
```

Set DATABASE_URL, JWT_SECRET, NEXT_PUBLIC_API_URL in .env

```
npx prisma generate
```

```
npm run dev
```

Production Deployment on Vercel

1. **Connect Repository:** Import the GitHub repository into Vercel.

2. **Set Environment Variables:** Add all required variables in Vercel project settings.
3. **Build Command:** `npm run build` runs `prisma generate` & next build.
4. **Database:** Use MongoDB Atlas. Set `DATABASE_URL` to the Atlas connection string.
5. **Stripe Webhooks:** Add webhook endpoint `/api/payments/webhook` in Stripe Dashboard.
6. **Deploy:** Push to main branch triggers automatic deployment.

Environment Variables Reference

Variable & Required & Description

`DATABASE_URL` & Yes
& MongoDB connection
string

`JWT_SECRET` & Yes &
JWT signing secret
(min 32 chars)

`NEXT_PUBLIC_API_URL`
& Yes & Base URL of
the application

`STRIPE_API_KEY` &
Optional & Stripe
secret key

`STRIPE_WEBHOOK_SECRET`
& Optional &
Stripe webhook signing
secret

`BREVO_API_KEY` &
Optional & Brevo email
API key

`SHIPPO_API_KEY` &
Optional & Shippo
shipping API key

`IMAGEKIT_PUBLIC_KEY`
& Optional &
ImageKit public key

`UPSTASH_REDIS_URL`
& Optional & Upstash
Redis URL

`OPENROUTER_API_KEY`
& Optional &
OpenRouter AI API key

Variable & Required & Description

SENTRY_DSN &
Optional & Sentry error
monitoring DSN

GOOGLE_CLIENT_ID &
Optional & Google
OAuth client ID

INTERNAL_API_KEY &
Optional & Bearer
token for cron
endpoints

Conclusion and Future Work

Conclusion

Stockly represents a comprehensive, production-ready solution to the challenges of modern inventory and supply chain management. The system successfully delivers on its core objectives: providing real-time stock visibility, automating order-to-invoice workflows, enforcing financial integrity, and delivering actionable business intelligence through integrated analytics and AI-assisted forecasting.

The architecture choices made throughout the project — Next.js 16 App Router for unified full-stack development, Prisma ORM for type-safe database access, JWT-based authentication with httpOnly cookies for security, and TanStack Query for efficient server state management — have proven effective in delivering a maintainable, scalable, and performant application.

- **Role-Based Architecture:** Three fully distinct user experiences (Admin, Supplier, Client) within a single application, with data isolation enforced at both API and database layers.
- **Financial Integrity:** Invoice calculations with negative-total prevention (`Math.max(0, ...)`), nullish coalescing for explicit zero values, and atomic stock reservation semantics.
- **Comprehensive Testing:** 107 documented white-box test cases across 10 modules, targeting 85% statement coverage and 90% branch coverage, with special emphasis on security-critical paths.
- **Extensible Integration Architecture:** Core application runs with only 3 environment variables; all integrations (Stripe, Shippo, Brevo, Redis, etc.) are optional and gracefully degraded.

- **Statistical Forecasting:** Demand forecasting engine using moving averages, standard deviation, linear regression, and z-score anomaly detection provides actionable reorder recommendations.
- **Production Deployment:** Successfully deployed on Vercel with MongoDB Atlas, achieving the target availability and performance metrics.

The white-box testing strategy documented in Chapter 5 provides a rigorous framework for validating the correctness of all critical system components. The test cases cover not only the happy paths but also edge cases, boundary values, error conditions, and security boundaries that are essential for a production system handling financial transactions and inventory data.

The SRS documented in Chapter 2 establishes clear, measurable requirements that guided the implementation and provide a baseline for future development. The functional requirements cover all major system domains, while the non-functional requirements address performance, security, scalability, maintainability, and auditability.

Lessons Learned

1. **Atomic Operations Matter:** The stock reservation logic revealed the importance of atomic database operations. The current implementation uses sequential Prisma updates, which introduces a theoretical race condition. In a high-concurrency production environment, MongoDB transactions should be used to ensure atomicity.
2. **Nullish Coalescing vs Logical OR:** The distinction between `??` and `||` in JavaScript is critical for financial calculations. Using `||` would incorrectly treat explicit zero values as falsy, leading to wrong tax/discount calculations. This was identified during white-box testing and is now explicitly documented.
3. **Environment Variable Security:** The JWT secret default fallback (`"your_jwt_secret"`) is a significant security risk that was identified during testing. Production deployments must always set `JWT_SECRET` to a cryptographically random value.
4. **Optional Integration Design:** Designing all integrations as optional from the start significantly improved developer experience and reduced the barrier to entry for new contributors. The graceful degradation pattern (check for config, return mock/disabled response if missing) should be applied consistently.
5. **Type Safety Pays Off:** TypeScript strict mode and Zod schema validation caught numerous potential runtime errors at compile time and during development. The investment in type safety significantly reduced debugging time.

Future Work

Short-Term Improvements (Next 3 Months)

1. **MongoDB Transactions:** Wrap order creation and stock reservation in a MongoDB transaction to eliminate the race condition and ensure atomicity.

2. **Atomic Order Number Generation:** Replace the count-based sequence with a Redis INCR atomic counter to prevent duplicate order numbers under high concurrency.
3. **Bounded Invoice Number Loop:** Add a maximum retry count to the generateInvoiceNumber while loop to prevent infinite loops under extreme load.
4. **JWT Secret Validation:** Add a startup check that throws an error if JWT_SECRET matches the default value in production environments.
5. **Comprehensive Test Suite:** Implement the 107 documented test cases using Vitest and achieve the target coverage thresholds.

Medium-Term Features (3–6 Months)

1. **Multi-Currency Support:** Extend the invoice and order models to support multiple currencies with real-time exchange rate conversion.
2. **Advanced Analytics Dashboard:** Implement more sophisticated business intelligence features including cohort analysis, customer lifetime value (CLV) calculation, and supplier performance scoring.
3. **Mobile Application:** Develop a React Native mobile application for warehouse staff to perform stock counts, receive shipments, and process orders on mobile devices.
4. **Barcode/QR Code Scanning:** Integrate barcode and QR code scanning in the mobile app for rapid product lookup and stock updates.
5. **Automated Reorder Workflows:** Implement automated purchase order generation when stock falls below the reorder threshold, with supplier notification via email.
6. **Advanced RBAC:** Implement fine-grained permissions (e.g., read-only admin, warehouse manager role) beyond the current 5-role system.

Long-Term Vision (6–12 Months)

1. **Multi-Tenant Architecture:** Evolve the platform into a multi-tenant SaaS product where multiple businesses can operate independently within the same infrastructure.
2. **Machine Learning Forecasting:** Replace the statistical forecasting engine with ML models (e.g., LSTM, Prophet) trained on historical sales data for more accurate predictions.
3. **EDI Integration:** Implement Electronic Data Interchange (EDI) for automated order exchange with large suppliers and retailers.
4. **IoT Integration:** Connect with IoT sensors for real-time warehouse monitoring (temperature, humidity, location tracking for perishable goods).

5. **Blockchain Audit Trail:** Explore blockchain-based immutable audit logging for high-value inventory items requiring regulatory compliance.
6. **GraphQL API:** Add a GraphQL API layer alongside the existing REST API to support more flexible client queries and reduce over-fetching.

Final Remarks

Stockly demonstrates that a modern full-stack application built with Next.js, TypeScript, Prisma, and MongoDB can deliver enterprise-grade functionality with a lean, maintainable codebase. The combination of strong typing, schema validation, role-based access control, and comprehensive white-box testing provides a solid foundation for a production system that handles real financial transactions and inventory data.

The open-source nature of the project (MIT license) and the comprehensive documentation in this report make it an excellent reference implementation for developers building similar systems. The architecture patterns, testing strategies, and integration approaches documented here are applicable to a wide range of full-stack web applications beyond inventory management.

The live demo at <https://stockly-inventory.vercel.app> demonstrates the system in action, and the source code is available for learning, reuse, and contribution.

API Endpoint Detailed Reference

Authentication Endpoints

POST /api/auth/register

Description: Register a new user account. Passwords are hashed with bcrypt (10 rounds) before storage.

Request Body:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "SecurePass123!",
  "username": "johndoe"
}
```

Validation Rules:

- name: Required, string, min 2 characters
- email: Required, valid email format, must be unique
- password: Required, string, min 8 characters

- username: Optional, string, must be unique if provided

Responses:

Status	Condition	Response Body
201	Success	{ "user": { id, name, email, role } }
400	Validation error	{ "error": "Validation failed", "details": [...] }
409	Email already exists	{ "error": "Email already registered" }
500	Server error	{ "error": "Internal server error" }

POST /api/auth/login

Description: Authenticate a user and set a JWT session cookie.

Request Body:

```
{
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Responses:

Status	Condition	Response Body
200	Success	{ "user": { id, name, email, role } } + Set-Cookie: session_id
400	Validation error	{ "error": "... " }
401	Invalid credentials	{ "error": "Invalid email or password" }
500	Corrupted user data	{ "error": "User data corrupted: password missing" }

Cookie Properties:

- httpOnly: true (prevents JavaScript access, mitigates XSS)

- secure: true in production (HTTPS only)
- sameSite: Strict (prevents CSRF)
- maxAge: 3600 seconds (1 hour, matches JWT expiry)
- path: / (accessible from all routes)

Product Endpoints

GET /api/products

Description: List products with optional filtering, sorting, and pagination. Results are role-scoped: suppliers see only their products.

Query Parameters:

Parameter	Type	Description
page	number	Page number (default: 1)
limit	number	Items per page (default: 20, max: 100)
search	string	Search by name or SKU
categoryId	string	Filter by category
supplierId	string	Filter by supplier
status	string	Filter by status (available/stock_low/out_of_stock)
sortBy	string	Sort field (name/price/quantity/createdAt)
sortOrder	string	Sort direction (asc/desc)

Response:

```
{
  "products": [
    {
      "id": "507f1f77bcf86cd799439011",
      "name": "Widget Pro",
      "sku": "WGT-PRO-001",
      "price": 29.99,
      "quantity": 150,
      "reservedQuantity": 10,
      "status": "available",
      "categoryId": "...",
    }
  ]
}
```

```

        "supplierId": "...",
        "imageUrl": "https://ik.imagekit.io/...",
        "createdAt": "2026-01-15T10:00:00.000Z"
    }
],
"total": 245,
"page": 1,
"limit": 20,
"totalPages": 13
}

```

POST /api/products

Description: Create a new product. Admin and user roles only.

Request Body:

```

{
  "name": "Widget Pro",
  "sku": "WGT-PRO-001",
  "price": 29.99,
  "quantity": 150,
  "categoryId": "507f1f77bcf86cd799439011",
  "supplierId": "507f1f77bcf86cd799439012",
  "status": "available",
  "description": "Professional grade widget",
  "expirationDate": "2027-12-31T00:00:00.000Z"
}

```

Order Endpoints

POST /api/orders

Description: Create a new order. Validates stock availability for all items before creating the order. Reserves stock atomically after creation.

Request Body:

```

{
  "clientId": "507f1f77bcf86cd799439013",
  "items": [
    {
      "productId": "507f1f77bcf86cd799439011",
      "quantity": 5
    },
    {
      "productId": "507f1f77bcf86cd799439014",
      "quantity": 2
    }
  ],
  "tax": 15.50,
}

```

```

    "shipping": 9.99,
    "discount": 5.00,
    "notes": "Please ship by Friday",
    "shippingAddress": {
      "street": "123 Main St",
      "city": "Springfield",
      "state": "IL",
      "zipCode": "62701",
      "country": "US"
    }
  }
}

```

Stock Validation Logic:

```

for (const item of data.items) {
  const product = await prisma.product.findUnique({ where: { id:
item.productId } });
  if (!product) throw new Error(Product not found: );

  const availableStock = Number(product.quantity) -
Number(product.reservedQuantity ?? 0);
  if (availableStock < item.quantity) {
    throw new Error(
      Insufficient stock for ". +
      Available: , Requested:
    );
  }
}

```

PATCH /api/orders/[id]

Description: Update order status. Triggers stock side-effects based on the status transition.

Status Transition Rules:

From	To	Stock Side-Effect
pending	confirmed	quantity -= qty, reservedQuantity - = qty
pending	cancelled	reservedQuantity - = qty (release reservation)
confirmed	cancelled	quantity += qty (restore stock)
processing	cancelled	quantity += qty (restore stock)
shipped	cancelled	quantity += qty (restore stock)

From	To	Stock Side-Effect
confirmed	processing	No stock change
processing	shipped	No stock change
shipped	delivered	No stock change

Database Schema Reference

Complete Prisma Schema

This appendix provides the complete annotated Prisma schema for reference.

User Model

```
model User {
  id          String      @id @default(auto()) @map("_id")
  @db.ObjectId
  createdAt   DateTime    @db.Date
  email       String      @unique
  name        String
  password    String      // bcrypt hashed, min 10 rounds
  updatedAt   DateTime?   @db.Date
  username    String?     @unique
  googleId    String?     // Google OAuth ID
  image       String?     // Profile image URL
  role        String?     // user | admin | supplier | client |
  retailer
  emailPreferences Json?   // Notification preferences
  notifications Notification[]
}
```

Product Model

```
model Product {
  id          String      @id @default(auto()) @map("_id")
  @db.ObjectId
  name        String
  sku         String      @unique // globally unique
  price       Float
  quantity    BigInt      // total physical stock
  reservedQuantity BigInt @default(0) // reserved by pending
  orders
  // availableStock = quantity - reservedQuantity (computed in
  // application)
  status      String      // available | stock_low | out_of_stock
  categoryId  String      @db.ObjectId
  supplierId  String      @db.ObjectId
  userId      String      @db.ObjectId
  imageUrl    String?
  imageFileId String?     // ImageKit file ID for cleanup
}
```

```

qrCodeUrl      String?
qrCodeFileId   String?
expirationDate DateTime? @db.Date
createdAt      DateTime @default(now()) @db.Date
updatedAt      DateTime? @db.Date
orderItems     OrderItem[]
}

```

Order and Invoice Models

```

model Order {
  id          String      @id @default(auto()) @map("_id")
  @db.ObjectId
  orderNumber String      @unique // ORD-YYYY-MMDD-HHMMSS-NNNN
  userId      String      @db.ObjectId
  clientId    String?     @db.ObjectId
  status      String      @default("pending")
  // pending | confirmed | processing | shipped | delivered |
  cancelled
  paymentStatus String    @default("unpaid")
  // unpaid | paid | refunded | partial
  subtotal    Float       @default(0)
  tax          Float?
  shipping     Float?
  discount     Float?
  total        Float       @default(0)
  shippingAddress Json?
  billingAddress Json?
  notes        String?
  trackingNumber String?
  trackingCarrier String?
  trackingUrl   String?
  labelUrl      String?
  estimatedDelivery DateTime?
  shippedAt      DateTime?
  deliveredAt    DateTime?
  cancelledAt    DateTime?
  stripePaymentIntentId String?
  createdAt      DateTime @default(now()) @db.Date
  updatedAt      DateTime? @db.Date
  items          OrderItem[]
  invoice        Invoice? // one-to-one, cascade delete
}

```

White-Box Testing: Additional Test Cases

Extended Authentication Tests

Route: POST /api/auth/register **Scenario:** Duplicate email registration

Input: { email: "existing@test.com", password: "pass", name: "Test" }

Mock: prisma.user.findUnique returns existing user
Expected: { error: "Email already registered" }, status **409**
Coverage: Duplicate email branch

Route: POST /api/auth/register **Scenario:** Password too short
Input: { email: "new@test.com", password: "abc", name: "Test" }
Expected: ZodError on password field, status **400**
Coverage: Password minimum length validation

Route: GET /api/auth/session **Scenario:** Valid session returns user
Mock: Valid session_id cookie, user exists in DB
Expected: Returns user object without password field, status **200**
Coverage: Session retrieval happy path; password field exclusion

Route: GET /api/auth/session **Scenario:** Response must not include password
Expected: Response object does NOT contain password field
Coverage: Sensitive field exclusion (security)

Extended Order Tests

Route: PATCH /api/orders/[id] **Scenario:** Cancel pending order releases reservation
Input: { status: "cancelled" }, order was pending
Mock: Order with 3 items, each with reservedQuantity = 5
Expected: Each product's reservedQuantity decremented by item quantity
Coverage: Cancel-from-pending stock release branch

Route: PATCH /api/orders/[id] **Scenario:** Cancel confirmed order restores stock
Input: { status: "cancelled" }, order was confirmed
Expected: Each product's quantity incremented by item quantity
Coverage: Cancel-from-confirmed stock restore branch

Route: PATCH /api/orders/[id] **Scenario:** Confirm order deducts stock
Input: { status: "confirmed" }, order was pending
Expected: Each product's quantity decremented AND reservedQuantity decremented
Coverage: Confirm order stock deduction branch

Route: DELETE /api/orders/[id] **Scenario:** Non-admin cannot delete order
Mock: session.role = "client"
Expected: { error: "Forbidden" }, status **403**
Coverage: Delete authorization guard

Extended Invoice Tests

Route: POST /api/invoices/[id]/send **Scenario:** Send invoice email
Mock: Invoice exists, Brevo configured, client has email
Expected: Invoice status updated to "sent", sentAt set, email dispatched
Coverage: Invoice send happy path

Route: POST /api/invoices/[id]/send **Scenario:** Invoice already paid
Mock: Invoice status = "paid"
Expected: { error: "Cannot send a paid invoice" }, status **400**
Coverage: Invalid status transition branch

Route: GET /api/invoices/[id]/pdf **Scenario:** Generate PDF for invoice
Mock: Invoice exists with all fields populated
Expected: Response with Content-Type: application/pdf header
Coverage: PDF generation path

Extended Forecasting Tests

Function: analyzeTrend **Scenario:** avgY is 0 (no sales)
Input: All daily sales = 0
Internal: avgY = 0, so percentageChange = 0 (division guard)
Expected: trend = "stable", percentageChange = 0
Coverage: Division by zero guard in trend analysis

Function: generateProductForecast **Scenario:** Fewer than 7 recent days
Input: Only 5 days of sales data
Internal: recentDays.length < 7, so uses averageDailySales directly
Expected: predictedDailySales = averageDailySales (no weighted blend)
Coverage: Insufficient recent data branch

Function: detectAnomalies **Scenario:** Drop anomaly with medium severity
Input: 30 days of sales, one day at 2.7 standard deviations below mean
Expected: Anomaly with type = "drop", severity = "medium" ($2.5 < zScore \leq 3$)
Coverage: Branch C2 + C5

Extended RBAC Tests

Route: GET /api/admin/counts **Scenario:** Supplier attempts admin endpoint
Mock: session.role = "supplier"
Expected: { error: "Forbidden" }, status **403**
Coverage: Admin-only route guard for supplier role

Route: POST /api/products **Scenario:** Client attempts product creation
Mock: session.role = "client"
Expected: { error: "Forbidden" }, status **403**
Coverage: Write operation role guard

Route: GET /api/audit-logs **Scenario:** Non-admin attempts audit log access
Mock: session.role = "user"
Expected: { error: "Forbidden" }, status **403**
Coverage: Audit log admin-only guard

Extended Database Tests

Scenario: Delete user cascades to notifications

Expected: All Notification records for the user are deleted

Coverage: User-Notification cascade delete

Scenario: Delete support ticket cascades to replies

Expected: All SupportTicketReply records for the ticket are deleted

Coverage: SupportTicket-Reply cascade delete

Scenario: AuditLog index performance

Expected: Queries on [userId], [action], [entityType], [createdAt] use indexes

Coverage: Index utilization verification

Scenario: ProductReview composite index

Input: Query by [orderId, productId, userId]

Expected: One review per order-product-user combination enforced

Coverage: Composite index on ProductReview

Extended Stripe Tests

Route: POST /api/payments/webhook **Scenario:** payment_intent.payment_failed event

Input: Valid Stripe event with type = "payment_intent.payment_failed"

Expected: Invoice status remains unchanged; notification sent to admin

Coverage: Payment failure event handler

Route: POST /api/payments/checkout **Scenario:** Invoice already paid

Mock: Invoice status = "paid"

Expected: { error: "Invoice already paid" }, status 400

Coverage: Already-paid invoice guard

Glossary

Term	Definition
App Router	Next.js 16 routing paradigm using the app/ directory with React Server Components
bcrypt	Password hashing algorithm using adaptive cost factor (salt rounds)
Branch Coverage	Testing metric measuring the percentage of code branches (if/else) executed
CORS	Cross-Origin Resource Sharing — HTTP mechanism for cross-domain requests
CRUD	Create, Read, Update, Delete —

Term	Definition
	basic database operations
httpOnly Cookie	Browser cookie inaccessible to JavaScript, mitigating XSS attacks
JWT	JSON Web Token — compact, URL-safe token for stateless authentication
MoSCoW	Prioritization method: Must Have, Should Have, Could Have, Won't Have
Nullish Coalescing	JavaScript ?? operator returning right operand only when left is null or undefined
ORM	Object-Relational Mapper — abstracts database queries into typed function calls
Prisma	TypeScript ORM for Node.js supporting MongoDB, PostgreSQL, MySQL, and others
RBAC	Role-Based Access Control — permissions based on user roles
Redis	In-memory data structure store used for caching and rate limiting
SRS	Software Requirements Specification — formal document of system requirements
SSR	Server-Side Rendering — HTML generated on the server per request
Statement Coverage	Testing metric measuring the percentage of code statements executed
TanStack Query	React library for server state management, caching, and background synchronization
Webhook	HTTP callback triggered by an external service event (e.g., Stripe payment)
White-Box Testing	Testing technique examining internal code logic, branches, and data flows
Zod	TypeScript-first schema validation

Term

Definition

library